
Desenvolvimento de Aplicações Web

Lab 04 - Introdução ao JavaScript

Ano Letivo: 2025/2026

JavaScript não é Java!!!

JavaScript e **Java** são linguagens de programação **completamente diferentes**, criadas por empresas diferentes, com propósitos diferentes. O nome “JavaScript” foi escolhido por razões de marketing na década de 1990, quando Java estava no auge da popularidade. **Não confundir!**

JavaScript não é parecido com Java!!!

Embora ambas as linguagens partilhem alguma sintaxe inspirada em C, as semelhanças terminam aí. JavaScript é uma linguagem de scripting dinâmica, interpretada, principalmente usada para adicionar interatividade a páginas web. Java é uma linguagem compilada, fortemente tipada, orientada a objetos, usada para aplicações empresariais e backend.

JavaScript não tem nada que ver com Java!!!

Se está a aprender JavaScript, **esqueça tudo** sobre Java (se souber). São paradigmas, ecossistemas e ferramentas completamente distintas. JavaScript é a linguagem **essencial** da Web moderna — domina o frontend, está presente no backend (Node.js), e é uma das linguagens mais populares do mundo.

1 Introdução

Até agora, construímos páginas web com **HTML** (estrutura) e **CSS** (apresentação visual). Mas as nossas páginas são **estáticas** — não respondem a interações do utilizador de forma dinâmica.

JavaScript é a linguagem que torna a Web **interativa**. Com JavaScript podemos:

- Responder a eventos (cliques, teclas, movimentos do rato)
- Alterar conteúdo e estilos da página dinamicamente
- Validar formulários antes de enviar dados
- Criar animações e efeitos visuais
- Comunicar com servidores (AJAX, fetch)
- Construir aplicações web complexas (Gmail, Google Maps, Facebook, etc.)

Neste laboratório, vamos dar os **primeiros passos** em JavaScript: variáveis, funções, eventos básicos. O foco principal no **DOM (Document Object Model)** será no Lab 05.

Contexto histórico: JavaScript foi criado em 1995 por Brendan Eich em apenas 10 dias para o browser Netscape Navigator. Desde então, tornou-se uma das linguagens mais importantes e utilizadas do mundo: <https://en.wikipedia.org/wiki/JavaScript>.

2 Objetivos

Neste laboratório iremos:

- Compreender os fundamentos de JavaScript: variáveis, tipos de dados, operadores
- Criar e invocar funções simples
- Manipular o DOM de forma básica (`getElementById`, eventos)
- Utilizar a Consola do browser para testar e identificar bugs no código
- Implementar um **contador de cliques** interativo
- Criar um **relógio digital** que atualiza automaticamente
- Integrar JavaScript na nossa Página Pessoal

3 Pré-requisitos

- Visual Studio Code instalado
- Um browser moderno com ferramentas de desenvolvimento (Chrome, Firefox, Edge, Safari)
- Pasta `PaginaPessoal` dos laboratórios anteriores (Labs 01-03)
- Conhecimentos básicos de HTML e CSS
- Noções gerais de programação (variáveis, condicionais, ciclos)

4 Conceitos-Chave

4.1 JavaScript: A Linguagem da Web

JavaScript é uma linguagem de programação interpretada, dinâmica, de alto nível, usada principalmente para adicionar comportamento e interatividade a páginas web.

Características principais:

- **Interpretada:** Executada diretamente pelo browser (não precisa compilação)
- **Dinamicamente tipada:** Tipos de variáveis determinados em tempo de execução
- **Orientada a objetos e funcional:** Suporta múltiplos paradigmas
- **Event-driven:** Responde a eventos do utilizador (cliques, teclas, etc.)

4.2 Tipos de Dados Primitivos

Tipo	Descrição	Exemplo
number	Números (inteiros e decimais)	42, 3.14, -7
string	Texto (entre aspas)	"Olá", 'JavaScript'
boolean	Verdadeiro ou falso	true, false
undefined	Variável sem valor atribuído	let x;
null	Ausência intencional de valor	let x = null;

Tabela 1: Tipos de dados primitivos em JavaScript

4.3 Declaração de Variáveis

Existem 3 formas de declarar variáveis em JavaScript:

- `var` — Forma antiga (evitar em código moderno)
- `let` — Declaração moderna para variáveis mutáveis
- `const` — Declaração para constantes (valores imutáveis)

```
var antiga = 10;           // Evitar
let mutavel = 20;          // Preferir
const constante = 30;      // Para valores fixos

mutavel = 25;              // OK
constante = 35;            // ERRO! Constantes nao mudam
```

Código 1: Declaração de variáveis

4.4 Operadores

Categoria	Operadores	Exemplo
Aritméticos	+, -, *, /, %	5 + 3 = 8
Relacionais	>, <, >=, <=	10 > 5 = true
Igualdade	===, !==	5 === 5 = true
Lógicos	&&, , !	true && false = false

Tabela 2: Operadores em JavaScript

Igualdade Estrita vs. Fraca

Use **sempre** === (igualdade estrita) em vez de == (igualdade fraca). A igualdade fraca realiza conversões automáticas que podem causar bugs:

```
5 == "5"    // true (conversao automatica)
5 === "5"   // false (tipos diferentes)
```

Boas práticas: Preferir === para evitar surpresas.

4.5 Funções

Funções são blocos de código reutilizáveis. Em JavaScript, existem várias formas de definir funções:

```
// 1. Declaracao tradicional
function somar(a, b) {
    return a + b;
}

// 2. Funcao anonima (atribuida a variavel)
let subtrair = function(a, b) {
    return a - b;
};

// 3. Arrow function (sintaxe moderna)
let multiplicar = (a, b) => a * b;

// Invocar funcoes
console.log(somar(5, 3));    // 8
console.log(subtrair(10, 4)); // 6
console.log(multiplicar(3, 7)); // 21
```

Código 2: Formas de definir funções

4.6 DOM (Document Object Model)

O **DOM** é uma representação estruturada do documento HTML que JavaScript pode manipular. Cada elemento HTML torna-se um **objeto** que podemos selecionar, modificar ou eliminar.

Métodos essenciais:

- `document.getElementById("id")` — Seleciona elemento pelo ID
- `document.querySelector("seletor")` — Seleciona elemento por seletor CSS

- `elemento.textContent` — Acede/modifica texto do elemento
- `elemento.style.propriedade` — Modifica estilos CSS
- `elemento.addEventListener("evento", funcao)` — Adiciona evento

```
// Selecionar elemento
let titulo = document.getElementById("titulo");

// Alterar conteudo
titulo.textContent = "Novo Titulo";

// Alterar estilo
titulo.style.color = "blue";
titulo.style.fontSize = "24px";
```

Código 3: Manipulação básica do DOM

5 Procedimento

Nota Importante

Ao trabalhar com JavaScript, a **Consola do browser** é um recurso valioso. Use F12 (Chrome/Edge/Firefox) ou Cmd+Option+C (Safari) para abrir as ferramentas de desenvolvimento e aceder à Consola. Poderá testar código JavaScript em tempo real e ver mensagens de debug com `console.log()`.

5.1 Tarefa 1: Preparar Ambiente JavaScript

1. Abra a pasta PaginaPessoal no Visual Studio Code
2. Na pasta scripts, crie um novo ficheiro: `main.js`
3. Abra `index.html` e adicione a referência ao script JavaScript no `<head>` com o atributo `defer`:

```
<head>
  ...
  <script src="scripts/main.js" defer></script>
</head>
```

4. No ficheiro `main.js`, adicione uma mensagem de teste:

```
console.log("JavaScript carregado com sucesso!");
```

5. Abra `index.html` num browser
6. Pressione F12 para abrir as ferramentas de desenvolvimento
7. Na aba "Console", verifique se a mensagem aparece

Atributo defer

O atributo defer no script garante que:

- O script é descarregado em **paralelo** com o HTML (não bloqueia o rendering)
- O script só **executa após** o HTML estar completamente carregado
- Mantém todos os recursos organizados no <head>
- É a **prática moderna recomendada** para scripts que manipulam o DOM

Alternativa antiga: Colocar o script antes de </body> também funciona, mas defer é superior porque permite download paralelo.

5.2 Tarefa 2: Variáveis e Operações Básicas

Vamos experimentar com variáveis, tipos de dados e operadores na Consola do browser.

1. Abra a Consola do browser (F12 → aba "Console")
2. Experimente os seguintes comandos (digite e pressione Enter):

```
// Declarar variaveis
let nome = "Maria";
let idade = 25;
let estudante = true;

console.log(nome);           // "Maria"
console.log(idade);          // 25
console.log(estudante);      // true

// Operacoes aritmeticas
let a = 10;
let b = 3;
console.log(a + b);          // 13
console.log(a - b);          // 7
console.log(a * b);          // 30
console.log(a / b);          // 3.333...
console.log(a % b);          // 1 (resto da divisao)

// Concatenacao de strings
let primeiroNome = "Joao";
let ultimoNome = "Silva";
let nomeCompleto = primeiroNome + " " + ultimoNome;
console.log(nomeCompleto);   // "Joao Silva"

// Template strings (sintaxe moderna)
let mensagem = `O ${primeiroNome} tem ${idade} anos.`;
console.log(mensagem);

// Operadores relacionais
console.log(10 > 5);          // true
console.log(3 === 3);         // true
console.log(5 !== 10);        // true
```

Template Strings

As **template strings** (com crases ```) permitem interpolar variáveis diretamente no texto usando `${variavel}`. São mais legíveis que concatenação com `+`.

5.3 Tarefa 3: Funções

Agora vamos criar funções reutilizáveis. Adicione o seguinte código ao ficheiro `main.js`:

```
// Funcao para saudar
function saudar(nome) {
  return "Ola, " + nome + "!";
}

// Testar funcoes na Consola
console.log(saudar("Ana"));           // "Ola, Ana!"
```

Código 4: Funções básicas

Recarregue a página e verifique os resultados na Consola.

5.4 Tarefa 4: Contador de Cliques

Vamos criar um contador interativo que incrementa cada vez que clicamos num botão.

5.4.1 Passo 1: Adicionar HTML

No `index.html`, adicione o seguinte código numa secção apropriada (por exemplo, após o menu de navegação):

```
<section id="contador-seccao">
  <h2>Contador de Cliques</h2>
  <p>Numero de cliques: <span id="contador">0</span></p>
  <button id="botao-incrementar">Clique Aqui!</button>
  <button id="botao-reset">Reiniciar</button>
</section>
```

5.4.2 Passo 2: Adicionar JavaScript

No ficheiro `main.js`, adicione o seguinte código:

```
// Esperar que o DOM esteja carregado
document.addEventListener('DOMContentLoaded', function() {

  // Selecionar elementos
  let contadorElemento = document.getElementById('contador');
  let botaoIncrementar = document.getElementById('botao-incrementar');
  let botaoReset = document.getElementById('botao-reset');

  // Variavel para guardar contagem
  let contagem = 0;

  // Funcao para incrementar contador
  function incrementar() {
    contagem++;
  }
```

```
        contadorElemento.textContent = contagem;
        console.log("Contador:", contagem);
    }

    // Funcao para reiniciar contador
    function reiniciar() {
        contagem = 0;
        contadorElemento.textContent = contagem;
        console.log("Contador reiniciado");
    }

    // Adicionar event listeners
    botaoIncrementar.addEventListener('click', incrementar);
    botaoReset.addEventListener('click', reiniciar);
});
```

Código 5: Contador de cliques

5.4.3 Passo 3: Testar

1. Recarregue a página no browser
2. Clique no botão “Clique Aqui!” várias vezes
3. Observe o número a incrementar
4. Clique em “Reiniciar” para voltar a zero
5. Verifique as mensagens na Console

DOMContentLoaded

O evento DOMContentLoaded garante que o JavaScript só executa **após** todo o HTML estar carregado. Isto evita erros ao tentar selecionar elementos que ainda não existem no DOM.

5.5 Tarefa 5: Relógio Digital

Vamos criar agora um relógio que mostra a hora atual e atualiza automaticamente a cada segundo.

5.5.1 Passo 1: Adicionar HTML

No `index.html`, adicione:

```
<section id="relogio-seccao">
  <h2>Relogio Digital</h2>
  <div id="relogio">00:00:00</div>
</section>
```


5.5.2 Passo 2: Adicionar CSS (opcional)

No ficheiro CSS (por exemplo, styles/main.css), adicione estilos para o relógio:

```
#relogio {  
  font-family: 'Courier New', monospace;  
  font-size: 48px;  
  font-weight: bold;  
  color: #333;  
  text-align: center;  
  padding: 20px;  
  background-color: #f0f0f0;  
  border: 2px solid #ccc;  
  border-radius: 10px;  
  display: inline-block;  
}
```

5.5.3 Passo 3: Adicionar JavaScript

No ficheiro main.js, adicione:

```
// Funcao para atualizar relógio  
function atualizarRelogio() {  
  // Obter hora atual  
  let agora = new Date();  
  
  // Extrair horas, minutos e segundos  
  let horas = agora.getHours();  
  let minutos = agora.getMinutes();  
  let segundos = agora.getSeconds();  
  
  // Adicionar zero a esquerda se necessario  
  horas = horas < 10 ? '0' + horas : horas;  
  minutos = minutos < 10 ? '0' + minutos : minutos;  
  segundos = segundos < 10 ? '0' + segundos : segundos;  
  
  // Formatar hora como string  
  let horaFormatada = `${horas}:${minutos}:${segundos}`;  
  
  // Atualizar elemento HTML  
  let relógioElemento = document.getElementById('relogio');  
  if (relógioElemento) {  
    relógioElemento.textContent = horaFormatada;  
  }  
}  
  
// Atualizar relógio de imediato  
atualizarRelogio();  
  
// Atualizar relógio a cada segundo (1000ms)  
setInterval(atualizarRelogio, 1000);
```

Código 6: Relógio digital

5.5.4 Explicação do Código

- `new Date()` — Cria objeto com data/hora atual
- `getHours()`, `getMinutes()`, `getSeconds()` — Extraem componentes da hora
- Operador ternário `? :` — Adiciona zero à esquerda se número < 10
- `setInterval(funcao, milissegundos)` — Executa função repetidamente

6 Conceitos Importantes Revisitados

6.1 Event Listeners

Event listeners permitem que o JavaScript “escute” eventos do utilizador (cliques, teclas, movimentos do rato, etc.) e execute código em resposta.

```
elemento.addEventListener('evento', funcaoCallback);
```

Eventos comuns:

Evento	Descrição
click	Clique com o rato
dblclick	Duplo clique
mouseenter	Rato entra no elemento
mouseleave	Rato sai do elemento
keydown	Tecla pressionada
keyup	Tecla libertada
submit	Formulário submetido
change	Valor de input alterado

Tabela 3: *Eventos comuns em JavaScript*

6.2 Debugging com Consola

A Consola é essencial para debug (encontrar e corrigir erros) no código JavaScript.

Métodos úteis:

```
console.log("Mensagem informativa");  
console.warn("Aviso");  
console.error("Erro");  
console.table([{nome: "Ana", idade: 25}, {nome: "Joao", idade: 30}]);  
console.clear(); // Limpar console
```

Breakpoints: Na aba “Sources” das ferramentas de desenvolvimento, pode adicionar breakpoints (pontos de paragem) no código para inspecionar variáveis passo a passo.

6.3 Separação de Responsabilidades

Boas práticas de estrutura:

- **HTML:** Estrutura e conteúdo
- **CSS:** Apresentação visual

- **JavaScript:** Lógica e comportamento

Evitar:

- JavaScript inline no HTML: <button onclick="...">
- CSS inline: style="color: red;"
- Manipular estilos diretamente em JavaScript (preferir adicionar/remover classes CSS)

Preferir:

```
// Em vez de:  
elemento.style.color = "red";  
elemento.style.fontSize = "20px";  
  
// Usar classes CSS:  
elemento.classList.add('destaque');
```

7 Resultado Esperado

No final deste laboratório, deve ter:

- ☐ Ficheiro scripts/main.js com código JavaScript funcional
- ☐ Contador de cliques interativo na página
- ☐ Relógio digital que atualiza automaticamente
- ☐ Código JavaScript bem organizado e comentado
- ☐ Mensagens de debug na Console do browser
- ☐ Compreensão dos conceitos: variáveis, funções, eventos, DOM
- ☐ Capacidade de testar e debuggar código JavaScript com a Console

8 Questões para Reflexão

1. Porque é importante usar === em vez de == em JavaScript?
2. Qual a diferença entre let e const? Quando usar cada um?
3. Qual a vantagem de usar o atributo defer nos scripts JavaScript?
4. O que acontece se tentar selecionar um elemento que não existe no DOM?
5. Como pode fazer debug a código JavaScript de forma eficaz?
6. Qual a vantagem de separar JavaScript do HTML (ficheiros externos)?

9 Próximos Passos

No próximo laboratório (Lab 05), iremos aprofundar a **manipulação do DOM**:

- Selecionar múltiplos elementos (`querySelectorAll`)
- Criar, modificar e remover elementos dinamicamente
- Manipular atributos e classes CSS
- Trabalhar com formulários e validação
- Eventos avançados e event delegation
- Animações e transições com JavaScript
- Trabalhar com estruturas de dados (arrays, objetos)

Recursos recomendados para estudo:

- Mozilla Developer Network (MDN): <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- JavaScript.info: <https://javascript.info/>
- Eloquent JavaScript (livro online gratuito): <https://eloquentjavascript.net/>

10 Referências

- **MDN Web Docs - JavaScript**: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- **JavaScript.info**: <https://javascript.info/>
- **Eloquent JavaScript**: <https://eloquentjavascript.net/>
- **ECMAScript Specification**: <https://tc39.es/ecma262/>
- **Can I Use**: <https://caniuse.com/> (compatibilidade de funcionalidades)
- **W3Schools JavaScript**: <https://www.w3schools.com/js/>